VECTOR

VHSM

Technical Reference

vHsm Custom Generic (CRYPTO)

Version 1.0.0

Authors	Tobias Finke
Status	Released

Document Information

History

Author	Date	Version	Remarks
Tobias Finke	2018-07-01	1.00.00	Initial Creation

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_CRYPTO.pdf	4.3.0
[2]	AUTOSAR	AUTOSAR_SWS_CSM.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_SWS_CRYIF.pdf	4.3.0

This technical reference describes the general use of the Crypto driver basis software.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 5

2 Introduction 6
2.1 Architecture Overview 7

3 Functional Description 9
3.1 Features 9
3.2 Initialization 9
3.3 Main Functions 9
3.4 Purpose of the Component 9

4 Integration 10
4.1 Scope of Delivery 10
4.1.1 Static Files 10
4.1.2 Templates – user modifiable 10

5 API Description 11
5.1 Services provided by VHSM_CUSTOM 11
5.1.1 vHsm_Custom_InitMemory 11
5.1.2 vHsm_Custom_Init 11
5.1.3 vHsm_Custom_MainFunction 12
5.1.4 vHsm_Custom_ProcessJob 13
5.1.5 vHsm_Custom_CancelJob 13
5.1.6 vHsm_Custom_KeyElementSet 14
5.1.7 vHsm_Custom_KeyElementGet 14
5.2 Services used by VHSM_CUSTOM 15

6 Glossary and Abbreviations 16
6.1 Glossary 16
6.2 Abbreviations 16

7 Contact 17

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 7
Figure 2-2 vHsm Architecture 8

Tables

Table 1-1 Component history 5
Table 4-1 Static files 10
Table 4-2 Templates 10
Table 5-1 vHsm_Custom_InitMemory 11
Table 5-2 vHsm_Custom_Init 11
Table 5-3 vHsm_Custom_MainFunction 12
Table 5-4 vHsm_Custom_MainFunction 13
Table 5-5 vHsm_Custom_CancelJob 13
Table 5-6 vHsm_Custom_KeyElementSet 14
Table 5-7 vHsm_Custom_KeyElementGet 15
Table 5-8 Services used by the VHSM_CUSTOM 15
Table 6-1 Glossary 16
Table 6-2 Abbreviations 16

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	Initial Creation
1.01	Release of component

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module VHSM_CUSTOM as specified in [1].

Supported AUTOSAR Release*:	4.3.0	
Supported Configuration Variants:	-	
Vendor ID:	VHSM_CUSTOM_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VHSM_CUSTOM_MODULE_ID	114 decimal (according to ref. [1])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The component (which is provided as a template) is located in the vHsm.

It acts like a CRYPTO driver which provides custom functionality to the CRYPTO driver on the application side.

The custom behavior can be implemented inside the functions described in Chapter 5.1 „Services provided by VHSM_CUSTOM“ by using custom key element ids and custom algorithms.

2.1 Architecture Overview

The following figure shows where the VHSM_CUSTOM is located in the AUTOSAR architecture.

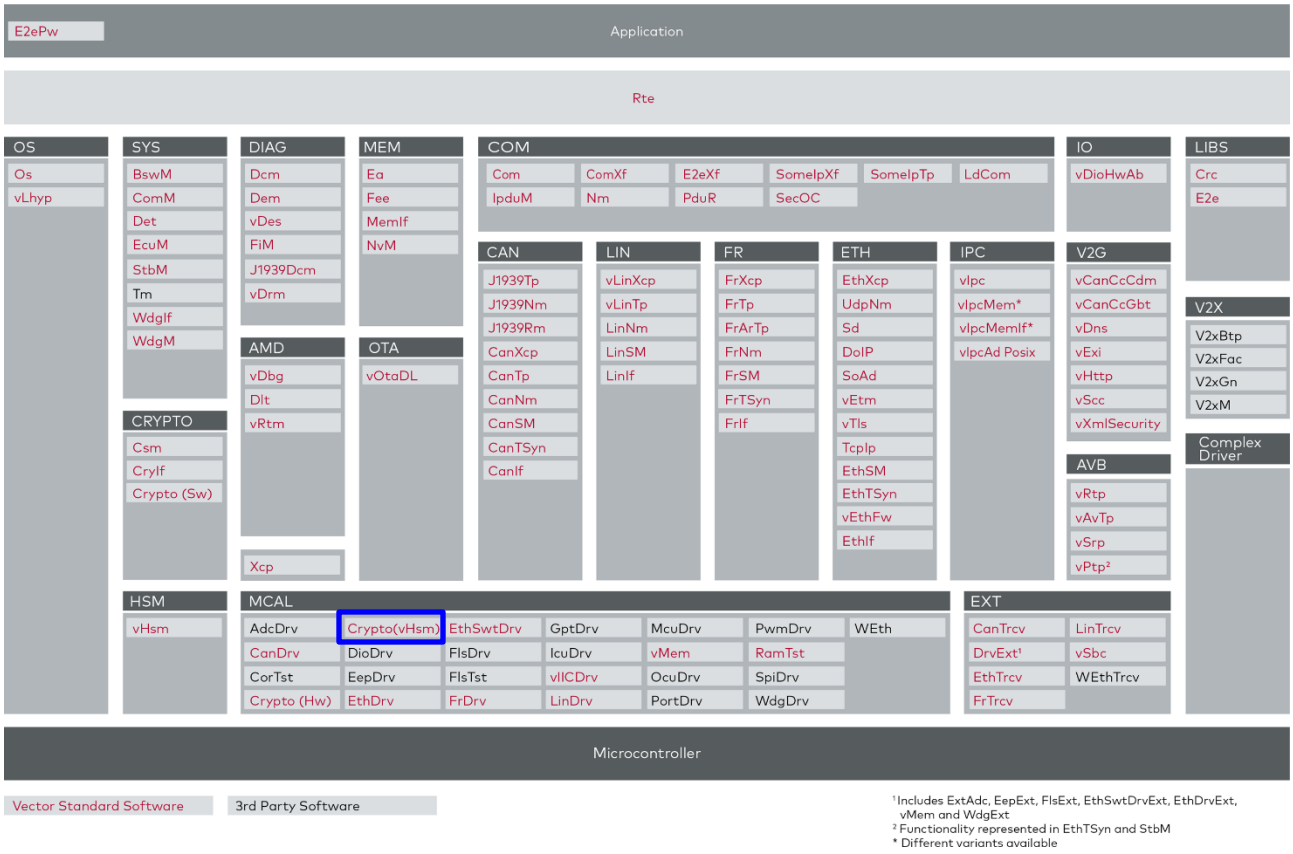


Figure 2-1 AUTOSAR 4.3 Architecture Overview

Expert Knowledge
The component is not located on the vHsm and acts like a CRYPTO driver which provides custom functionality to the CRYPTO driver on the application side.

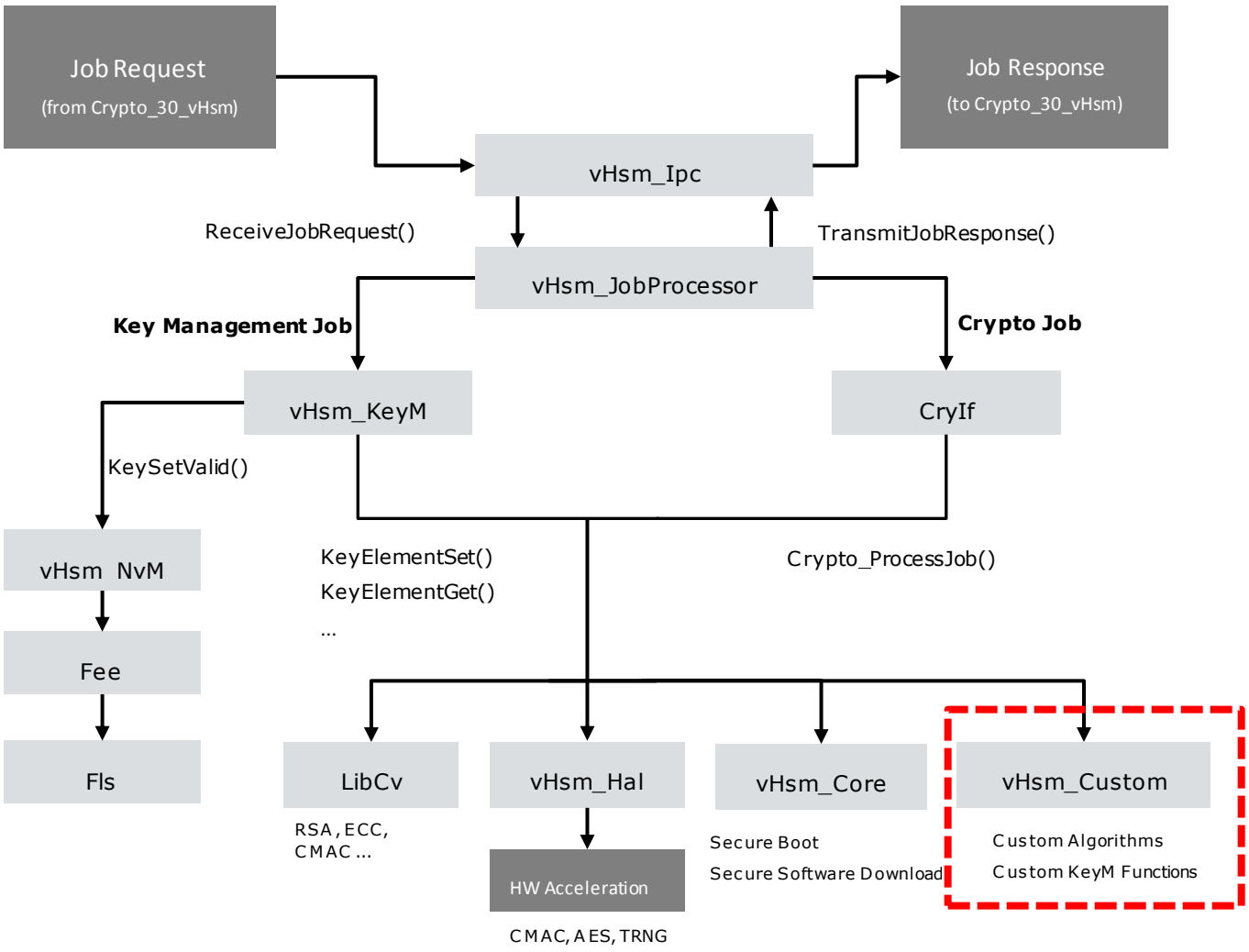


Figure 2-2 vHsm Architecture

3 Functional Description

3.1 Features

The VHSM_CUSTOM has the same API as an AUTOSAR CRYPTO driver specified in [1]. The functionality is specific to the actual implementation of the customer as the provided files are only templates.

3.2 Initialization

The vHsm Firmware calls the function vHsm_Custom_InitMemory() and the vHsm_Custom_Init() during the initialization phase of the vHsm.

3.3 Main Functions

The function vHsm_Custom_MainFunction() is called cyclically by the vHsm.

3.4 Purpose of the Component

The component gives the user the flexibility to extend the vHsm firmware with own functionality.

- Add special algorithms which are not supported by the Crypto_30_LibCv driver

  (e.g. CRYPTO_ALGOFAM_CUSTOM_VHSM_SECURE_BOOT)
- Add special behavior by using custom key element ids

  (e.g. CRYPTO_KE_CUSTOM_VHSM_VERSION)

The already existing drivers can be used as reference for implementing custom functionalities.

- vHsm_Hal
- vHsm_Core



Caution
The APIs available inside the vHsm may be subject to changes.
Future versions of the vHsm will provide a defined set of APIs available for the Custom CRYPTO driver.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR VHSM_CUSTOM into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the VHSM_CUSTOM contains the files which are described in the chapters 4.1.1:

4.1.1 Static Files

File Name	Description
vHsm_Custom.h	This is the header file of the driver
vHsm_Custom_Custom.h	This file contains the custom definitions for custom key element ids or custom algorithms.
Crypto_30_vHsm_Custom.h	This header file provides necessary defines for the Crylf.

Table 4-1 Static files

4.1.2 Templates – user modifiable

File Name	Description
_vHsm_Custom.c	This is the source file of the Driver. It contains the interfaces needed by the vHsm Core. The file can be edited by the user.

Table 4-2 Templates



Edit
The functions inside the template files can be modified by the user to his needs.

5 API Description

The following chapters describe the interfaces which must to be provided to the vHsm Firmware.

5.1 Services provided by VHSM_CUSTOM

5.1.1 vHsm_Custom_InitMemory

Prototype
`void` **`vHsm_Custom_InitMemory`** `( void )`
Functional Description
Initialize the memory.
Particularities and Limitations
> This function is synchronous. > This function is non-reentrant.
Expected Caller Context
> Task

Table 5-1 vHsm_Custom_InitMemory

5.1.2 vHsm_Custom_Init

Prototype
`void` **`vHsm_Custom_Init`** `( void )`
Functional Description
Initialize the driver. **Caution** It is not guaranteed that the other modules in the vHsm are initialized when this function is called. E.g. the KeyStorage cannot be used during this stage.
Particularities and Limitations
> This function is synchronous. > This function is non-reentrant.
Expected Caller Context
> Task

Table 5-2 vHsm_Custom_Init

5.1.3 vHsm_Custom_MainFunction

Prototype
`void` **`vHsm_Custom_MainFunction`** `( void )`
Functional Description
Initialize the driver. **Caution** The vHsm does not have scheduling. When a job is sent to the vHsm which will take a long time, the vHsm_Custom_MainFunction() will not be triggered for the time the job is processing.
Particularities and Limitations
> This function is synchronous. > This function is non-reentrant.
Expected Caller Context
> Task

Table 5-3 vHsm_Custom_MainFunction

5.1.4 vHsm_Custom_ProcessJob

Prototype	
`Std_ReturnType` **`vHsm_Custom_ProcessJob`** `( uint32 objectId, Crypto_JobType *job )`	
Parameter	
`objectId`	Id of the driver object
`job`	Pointer to the job structure
Return code	
`E_OK`	If request was successful
…	…
Functional Description	
This function is called when the Crypto_30_vHsm CRYPTO driver on the application receives a job for the custom driver object. This job is then passed to this function. It is possible to implement special behavior by using and defining custom algorithm modes and families.	
Particularities and Limitations	
> This function is synchronous.	
> This function is non-reentrant.	
Expected Caller Context	
> Task	

Table 5-4 vHsm_Custom_MainFunction

5.1.5 vHsm_Custom_CancelJob

Prototype	
`Std_ReturnType` **`vHsm_Custom_CancelJob`** `( uint32 objectId, Crypto_JobType *job )`	
Parameter	
`objectId`	Id of the driver object
`job`	Pointer to the job structure
Return code	
`E_OK`	Request successful, job has been removed.
`E_NOT_OK`	Request failed, job could not be removed.
Functional Description	
This function sis called when the Crypto_30_vHsm CRYPTO driver on the application receives cancel request for a job on the custom crypto driver.	
Particularities and Limitations	
> This function is synchronous.	
> This function is non-reentrant.	
Expected Caller Context	
> Task	

Table 5-5 vHsm_Custom_CancelJob

5.1.6 vHsm_Custom_KeyElementSet

Prototype	
`Std_ReturnType` **`vHsm_Custom_KeyElementSet`** `( uint32 cryptoKeyId, uint32 keyElementId, uint8 * keyPtr, uint32 keyLength )`	
Parameter	
`cryptoKeyId`	Id of the key
`keyElementId`	Id of the key element
`keyPtr`	Pointer to the data which is provided by the application
`keyLength`	Length of the data which is provided by the application
Return code	
`E_OK`	Request successful
…	…
Functional Description	
This function sis called when the Crypto_30_vHsm CRYPTO driver on the application receives a request to set data on the key which is reverenced as custom key in the vHsm key mapping container.	
Particularities and Limitations	
> This function is synchronous.	
> This function is non-reentrant.	
Expected Caller Context	
> Task	

Expert Knowledge
To implement custom behavior it is recommended to specify and use own key element ids.

It can then be checked if a custom key element id has been requested and otherwise relay the request to the LibCv.

Table 5-6 vHsm_Custom_KeyElementSet

5.1.7 vHsm_Custom_KeyElementGet

Prototype	
`Std_ReturnType` **`vHsm_Custom_KeyElementGet`** `( uint32 cryptoKeyId, uint32 keyElementId, uint8 * resultPtr, uint32 resultLengthPtr )`	
Parameter	
`cryptoKeyId`	Id of the key
`keyElementId`	Id of the key element
`keyPtr`	Pointer to the memory location where the result shall be stored

keyLength	Length of the buffer provided by the application to store the result. After calling this function, the number of written bytes shall be stored to the pointer.
Return code	
E_OK	Request successful
…	…
Functional Description	
This function sis called when the Crypto_30_vHsm CRYPTO driver on the application receives a request to get data on the key which is reverenced as custom key in the vHsm key mapping container. **Expert Knowledge** To implement custom behavior it is recommended to specify and use own key element ids. It can then be checked if a custom key element id has been requested and otherwise relay the request to the LibCv.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant.	
Expected Caller Context	
> Task	

Table 5-7 vHsm_Custom_KeyElementGet

5.2 Services used by VHSM_CUSTOM

In the following table services provided by other components, which are used by the VHSM_CUSTOM are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
vHsm	vHsm_KeyM_WriteOnceHandling_KeyElementSet
Crypto_30_LibCv	Crypto_30_LibCv_KeyElementGet

Table 5-8 Services used by the VHSM_CUSTOM

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
HSM	A special core dedicated for secure execution of code with hardware modules available to accelerate calculations necessary for crypto primitives like AES.

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification
HSM	Hardware security module

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com